

Decision Trees and Ensemble Methods

Machine Learning

Decision Trees

- A decision tree is a supervised machine learning model used both for classification or regression tasks
- It uses binary decisions on features to take decisions
- A decision tree consists of:
 - a root node (representing the entire datasets)
 - internal nodes (where data are divided using binary decisions)
 - leaf (terminal) nodes (where the final prediction is made)
- The model splits data step-by-step with the aim to create groups that are as homogeneous as possible
 - in classification = terminal nodes that includes samples of the same class
 - in regression = terminal nodes that includes samples with similar output
- Predictions are extremely fast
- Decision Trees can easily handle both numerical and categorical data

Cost Function – For Regression

- Mean Squared Error (contribution for each leaf node)

$$J = \frac{1}{m_k} \sum_i (y_i - \bar{y}_k)^2$$

number of samples in the leaf node

average of the output in the leaf node

sum over the samples inside the leaf node

- Mean Absolute Error

$$J = \frac{1}{m_k} \sum_i |y_i - \bar{y}_k|$$

Prediction = average in the leaf node

Cost Function – For classification

Gini impurity for one leaf node

$$G = \sum_k p_k \sum_{i \neq k} p_i = \sum_k p_k (1 - p_k) = \sum_k p_k - p_k^2 = 1 - \sum_k p_k^2$$

sum over the classes

probability of being in another class (in that node)

probability of being in class k (in that node)

Gini Impurity. It quantifies the probability that a randomly chosen sample would be incorrectly classified if it were labelled according to the class distribution in that node.

- $G = 0 \rightarrow$ The node includes only one class
- $G = 0.5 \rightarrow$ In case of two classes, it means the node includes the same number of samples for each of them

Prediction = most common class in the leaf node (in case of impure node, the fraction of training samples in the class can be used to estimate its probability)

CART (Classification and Regression Tree) algorithm

If we consider only binary features and only 2 classes, there are 2^{2^n} possible ways to label the data → an exhaustive search of the space is not possible

CART constructs a tree by recursively splitting the dataset into subsets based on feature values, with the goal of making each subset as homogeneous as possible.

- For classification → purity = how homogeneous the class labels are (usually measured as Gini impurity)
- For regression → purity = how similar the numeric values are (usually measured as MSE)

Algorithm

1. Define a root node that includes all the samples in the dataset
2. For every feature and possible split threshold (i.e.: all values between two sorted samples) evaluates how well the split separates the data
 - for classification it used the Gini impurity
 - for regression the MSE
3. Select the best splitting
4. Divide the dataset into two subsets
5. Go back to point 2 until
 - the nodes are pure
 - a stopping condition is reached (for example: maximum depth of the tree, minimum number of samples in the node, maximum number of leaf nodes)

$$G_{split} = \frac{m_L}{m} G(L) + \frac{m_R}{m} G(R)$$

number of samples in the left node

impurity of the left node

Cost complexity pruning

The final complete tree might cause overfitting. With pruning we try to balance the fit quality with the model complexity, and consequently to reduce overfitting.

- Compute α_t for all the internal nodes
- Find the node with the minimal value of α_t . Since the numerator is the change in training error, and the denominator is the change in complexity, we are selecting the node that gives a minimal contribution with respect to its complexity.
- Prune that subtree and repeat the operation
- In this way a series of nested tree is created
- Use cross-validation to select the optimal value of alpha (and consequently the optimal tree)

training error of the tree

(misclassification error or MSE)

training error of the full tree

$$\alpha_t = \frac{R(T) - R(T_t)}{|T_t| - 1}$$

number of leaf
nodes in full
tree

because if we prune the
tree the complexity is
reduced by 1.
The denominator is the
reduction in complexity

Ensemble of models

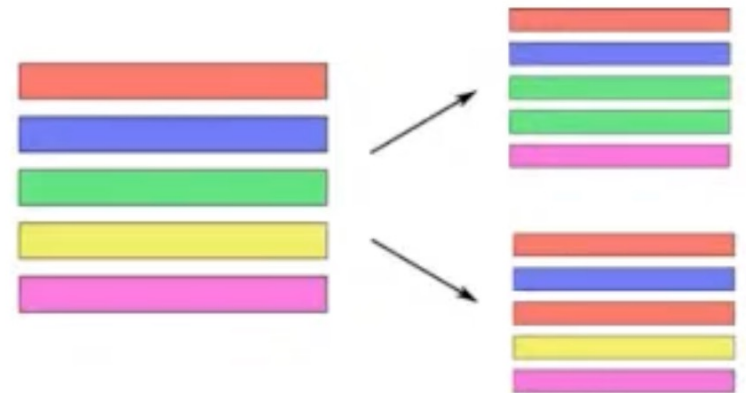
Core idea: more models are better than one model. So instead of creating a single tree, let's create numerous tree and then combine their predictions.

The predictions (classification) can be combined using:

- Soft-voting = average of the probabilities
- Hard-voting = select the most voted class

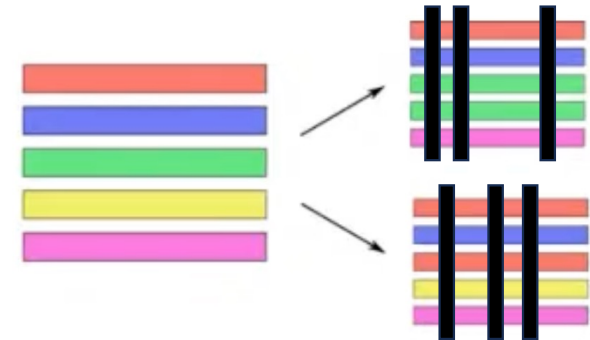
Bagging (Bootstrap Aggregating)

- Different predictors (same algorithm) are trained using different datasets generated by Bootstrap with replacement
- Train one decision tree per dataset
- Aggregate the predictions



Random Forest

- Ensemble of models work better when the models are less correlated
- A possible strategy to make the model less correlated is to train the models using, not only different set of samples, but also different set of features.
- Create multiple datasets by sampling the original training dataset with replacement
- For each dataset remove some of the features
- Train one decision tree per dataset
- Aggregate the predictions



ADABOOST (Adaptive Boosting)

Instead of taking the samples randomly as in bagging, in boosting the model focuses on the samples that are hard to classify.

- Initialize weight for all samples equal to $1/\text{number_of_samples}$
- For $k = [1:k_max]$:

- select samples according to weights
- train a weak learner
(2 classes model)
- compute the weight error (epsilon)
- compute the model weight (alpha)
- update weights
- normalize weights to 1

- Predict one if $F > 0$

$$F = \sum_k \alpha_k h_k(x)$$

prediction of the weak learner $\{-1, 1\}$

$$\epsilon = \sum_i w_i \delta(h(x_i), y_i)$$

1 if the prediction is wrong
sum over the samples

$$\alpha_k = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right)$$

high for good models

$$w_i = \begin{cases} w_i e^{-\alpha_k} & \text{for samples correctly classified} \\ w_i e^{\alpha_k} & \text{for samples incorrectly classified} \end{cases}$$

The weight increases if the sample is misclassified

Gradient Boosting

Instead of reweighting data, like in AdaBoost, gradient boosting fits each new model to the residual errors.

- Train a weak learner: $f_1(x)$
- for $k = [1:k_max]$
 - Compute the derivative of the loss as a function of the hypothesis (pseudo-residuals)
 - Train a new weak learner to predict the residuals
 - Update the model

$$F(x) = \sum_k \gamma_k h_k(x)$$

$$r_i^k = - \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

$$\gamma_k = \operatorname{argmin}_{\gamma} \left(\sum_i L(y_i, F(x_i) + \gamma h_k(x_i)) \right)$$

$$F(x) = F(x) + \gamma_k h_k(x)$$

for regression

$$L_i = (y_i - F(x_i))^2$$

$$r_i = -\frac{\partial L_i}{\partial F(x_i)} = y_i - F(x_i)$$

for classification

$$y_i \in \{-1, 1\}$$

$$P_i(y_i|x_i) = \frac{1}{1 + e^{-y_i F(x_i)}}$$

$$L_i = -\log P_i = \log(1 + e^{-y_i F(x_i)})$$

$$r_i = -\frac{\partial L_i}{\partial F(x_i)} = -\frac{-e^{-y_i F(x_i)}}{1 + e^{-y_i F(x_i)}} = \frac{y_i}{1 + e^{y_i F(x_i)}}$$

$$y_i = 1 \rightarrow r_i = \frac{1}{1 + e^{F(x_i)}} = 1 - \frac{1}{1 + e^{-F(x_i)}}$$

$$y_i = -1 \rightarrow r_i = -\frac{1}{1 + e^{-F(x_i)}}$$

it's large
when the
model is
wrong